

## Descripción Fundamentada de la Lógica del Negocio

La lógica del negocio de nuestro Sistema de Gestión Penitenciaria (SGP) gira en torno a un ciclo de vida central: **el expediente del recluso**. Todas las operaciones del sistema (seguridad, ubicación, visitas, personal) interactúan con este expediente y deben garantizar la integridad de los datos y el cumplimiento de los protocolos legales. Para lograrlo, el sistema implementa un control de acceso basado en roles .

### 1. Módulo de Registro e Ingreso (Admisión)

Este será el "punto de entrada" de la información y el control legal. La lógica aquí es estricta y secuencial.

**Lógica Principal:** Un recluso no existe en el sistema hasta que se verifica una orden judicial válida. El sistema debe asegurarse bien de los datos del recluso para evitar suplantaciones de identidad.

**Flujo:** Verificación Legal -> Registro Biométrico -> Clasificación

**Inicial.** Esta clasificación (ej. alta o baja peligrosidad) es fundamental, ya que determinará automáticamente las opciones disponibles en el módulo de ubicación.

### 2. Módulo de Control de Ubicación en celdas:

Este componente automatiza la asignación de espacios basándose en los criterios de peligrosidad definidos previamente. En este se gestiona la logística física y la seguridad interna. Su lógica se basa en reglas de compatibilidad del preso y capacidad de la celda que se le asigne.

**Lógica Principal:** El sistema debe impedir asignaciones de celda que violen reglas de seguridad (ej. no mezclar reclusos de alta peligrosidad con los de baja, o miembros de bandas rivales). Además, debe gestionar alertas de hacinamiento.

**Flujo:** Consulta de Capacidad Real-> Validación de Reglas de Seguridad -> Asignación/Traslado de Celda. Cada movimiento físico debe quedar registrado históricamente en el expediente del recluso.

### 3. Módulo de Seguridad y Visitas:

Este componente centraliza el control de acceso y la vigilancia perimetral en el recinto penitenciario,

Este módulo regula la interacción del mundo exterior con el interior de la penitenciaría, equilibrando derechos con la seguridad del penal.

**Lógica Principal:** La lógica aquí es de autorización y control. No cualquier persona puede visitar; deben ser registradas y aprobadas previamente (ej. familiares directos, abogados). El sistema debe controlar las asistencias a las salas de visita y los tiempos de duración

**Flujo:** Registro de Visitante->Vinculación con Recluso (Parentesco/Rol) -> Programación de Turno (Validando Aforo) -> Control de Acceso (Entrada/Salida).

#### **4. Módulo de Gestión de Personal**

Este módulo asegura que haya el personal adecuado con los permisos correctos para operar el sistema y el penal.

**Logica Principal:** Se basa en una jerarquía de roles y permisos (RBAC - Role-Based Access Control). Un custodio no debe tener permisos para modificar una condena; un registrador legal no debe gestionar turnos de guardia. Además, rastrea la responsabilidad de las acciones.

**Flujo:** Registro de Empleado -> Asignación de Rol (Custodio, Médico, Admin) -> Gestión de Turnos/Asignación de Áreas -> Auditoría de Acciones.

#### **5. Módulo de Reportes y Analítica**

## **CASOS DE USO DEL SISTEMA:**

### **Identificación de Actores**

**Encargado de Admisión :** Responsable del ingreso legal y los datos “EXATOS” del recluso. Registro de datos biométricos, historial judicial y asignación de celda inicial.

**Jefe de Seguridad:** Responsable de la asignación estratégica de celdas, traslados y la aprobación de las reglas de seguridad. Gestión de Incidentes y Reportes: Supervisa las alertas generadas por el sistema,

**Encargado de Visitas / Custodio de Acceso:** Encargado de registrar, verificar y controlar el ingreso del personal civil (visitantes).y validación de identidad mediante escaneo de documentos

**Administrador del Sistema / RRHH:** Gestiona el alta del personal, la asignación de roles y turnos de guardia.

## **2. Estructura de Casos de Uso por Módulo:**

### **Módulo 1: Registro e Ingreso (Admisión)**

Este bloque se enfoca en el ciclo inicial del recluso.

#### **CU01: Verificar Orden Judicial**

*Descripción:* El sistema permite validar que el documento legal de internamiento sea auténtico y vigente antes de habilitar los siguientes campos.

#### **CU02: Registrar Datos (Biométricos?)**

*Descripción:* Captura de huellas dactilares y fotografía digital del recluso.

*Relación:* <<include>> (Obligatorio) en el caso de uso

### **CU03: Crear Expediente Digital**

*Descripción:* Genera la ficha central del recluso en el *La base de Datos SGP (sistema de gestion penitenciaria)* con sus datos personales, delitos y condena.

### **CU04: Clasificar Perfil de Riesgo**

*Descripción:* Evaluación del nivel de peligrosidad (Alta, Media, Baja) basado en el historial y el delito.

## **Módulo 2: Control de Ubicación del preso**

Regulado por las alertas de capacidad y perfiles de riesgo determinados en el módulo 1.

### **CU05: Consultar Capacidad de Pabellones**

*Descripción:* Vista en tiempo real del mapa de celdas y su estado de ocupación para evitar el hacinamiento o asignarlo en una celda sin espacio..

### **CU06: Asignar Celda Inicial**

*Descripción:* El sistema sugiere automáticamente celdas disponibles que coincidan con el perfil de riesgo del recluso.

### **CU07: Programar Traslado Interno**

*Descripción:* Registro de movimientos de un recluso de un pabellón a otro por razones disciplinarias, médicas o edilicias.

*Relación:* <<extend>> (Opcional/Condicional) -> *Emitir Alerta de Incompatibilidad* (si se intenta juntar a reclusos rivales).CU08: Monitoreo de Ubicación en Tiempo Real

## **Módulo 3: Seguridad y Visitas**

Controla el flujo externo de personas asegurando el vínculo con el recluso.

#### **CU08: Registrar Visitante**

*Descripción:* Alta en el sistema de los datos del ciudadano que desea ingresar al penal.

#### **CU09: Verificar Parentesco y Antecedentes**

*Descripción:* Validación de que el visitante esté autorizado en la cartilla del recluso y no posea restricciones legales.

#### **CU10: Programar Turno de Visita**

*Descripción:* Agenda la cita en un día y hora específicos controlando el aforo de la sala de visitas.

#### **CU11: Controlar Acceso (Entrada/Salida)**

*Descripción:* Registro del momento exacto en que el visitante cruza la entrada al penal a hacer una visita y el momento en que se retira.

### **Módulo 4: Gestión de Personal**

El soporte de seguridad y roles que hará funcionar el penal.

#### **CU12: Registrar Personal Penal**

*Descripción:* Alta de custodios, médicos y administrativos.

#### **CU13: Asignar Roles y Permisos**

*Descripción:* Configuración del perfil de acceso del empleado dentro del sistema de Gestion Penitenciaria (ej. un Custodio no puede editar un Expediente Judicial).

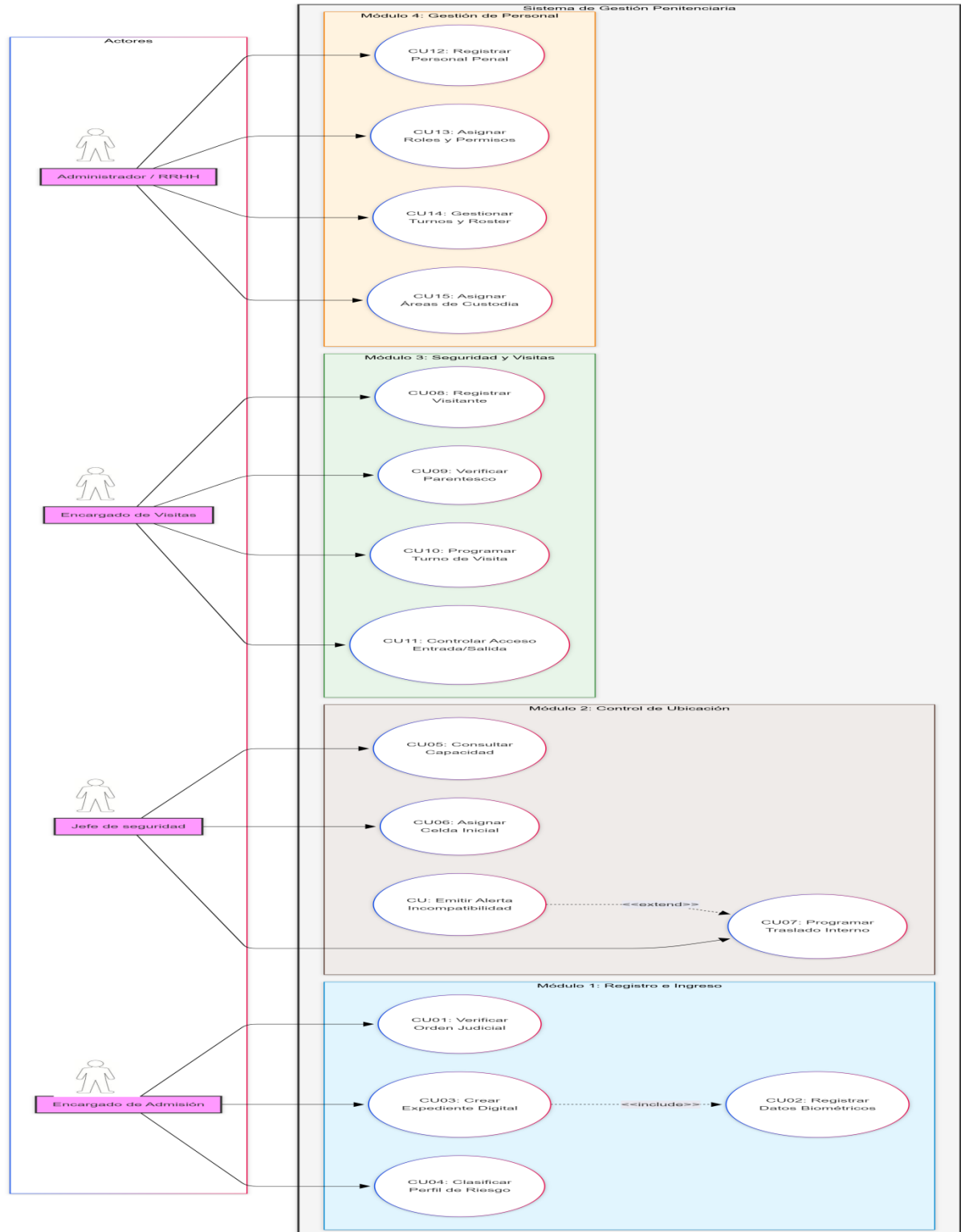
#### **CU14: Gestionar Turnos de custodios**

*Descripción:* Planificación de los horarios de guardia del personal de seguridad.

#### **CU15: Asignar Tareas y Áreas de Custodia**

*Descripción:* Designación de qué custodios vigilarán qué pabellones durante un turno específico.

## DIAGRAMA DE CASOS DE USO.



## MODELO DE CLASES DEL SISTEMA (UML)

### Módulo 1: Registro e Ingreso (Admisión)

- ✓ **Recluso:** Representa al interno dentro del ciclo de software.

| <i>Atributos</i>                                                                                                                                                       | <i>Métodos</i>                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| - idRecluso: int,<br>- nombres: String,<br>- apellidos: String,<br>- cedula: String,<br>- fechaNacimiento: Date,<br>- estadoRiesgo: String,<br>- estadoRecluso: String | + registrarIngreso(): boolean,<br>+ actualizarEstado(nuevoEstado: String): void,<br>+ obtenerPerfilRiesgo(): String |

- ✓ **ExpedienteJudicial:** Contiene la información legal. Tiene una relación de Asociación 1:1 con Recluso.

| <i>Atributos</i>                                                                                                                                                              | <i>Métodos</i>                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| - idExpediente: int,<br>- idRecluso: int,<br>- numeroCausa: String,<br>- delitoPrincipal: String,<br>- fechaIngreso: DateTime,<br>- aniosCondena: int,<br>- mesesCondena: int | + registrarExpediente(): void,<br>+ actualizarSentencia(nuevaSentencia: String): void,<br>+ calcularFechaLiberacion(): Date.<br>+ validarOrdenJudicial(): boolean,<br>+ calcularFechaLiberacion(): Date |

- ✓ **Biometria:** Datos de identidad estrictos. Relación de **Composición 1:1** con Recluso (si el recluso no se registra en el sistema, su biometría tampoco se guarda aislada).

| <i>Atributos</i> | <i>Métodos</i> |
|------------------|----------------|
|------------------|----------------|

|                                                                        |                                                                                                                                                                     |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| - idBiometria: int,<br>- huellaRaw: byte[],<br>- fotoPerfilUrl: String | + verificarIdentidad(huellaComparar: byte[]): boolean<br>+ registrarDatosBiometricos(): void,<br>+ actualizarHuella(): void<br>+ eliminarRegistroBiometrico(): void |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Módulo 2: Control de Ubicación.

- ✓ **Pabellon:** Estructura física mayor. Tiene una relación de **Composición 1** -> \* con la celda del preso.

| <i>Atributos</i>                                                                 | <i>Métodos</i>                   |
|----------------------------------------------------------------------------------|----------------------------------|
| - idPabellon: int,<br>- nombrePabellon: String,<br>- tipoRiesgoPermitido: String | + verificarCapacidadTotal(): int |

- ✓ **Celda:** Espacio de confinamiento. Tiene una relación de **Agregación 1** -> \* con Recluso (los reclusos se asignan a la celda, pero si la celda cambia, el objeto Recluso sigue existiendo).

| <i>Atributos</i>                                                                             | <i>Métodos</i>                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| - idCelda: int,<br>- codigoCelda: String<br>- capacidadMaxima: int,<br>- cantidadActual: int | + tieneEspacio(): boolean,<br>+ verificarIncompatibilidad(recluso: Recluso): boolean:<br>boolean,<br>+ asignarRecluso(recluso: Recluso): void,<br>+ liberarEspacio(recluso: Recluso): void |

- ✓ **HistorialUbicacion:** Clase asociativa que audita los movimientos.

| <i>Atributos</i>                                   | <i>Métodos</i> |
|----------------------------------------------------|----------------|
| -idHistorial: int,<br>- fechaAsignacion: DateTime, |                |



|                                                    |  |
|----------------------------------------------------|--|
| - fechaSalida: DateTime,<br>- motivoCambio: String |  |
|----------------------------------------------------|--|

### Módulo 3: Seguridad y Visitas

- ✓ **Visitante:** Civil que solicita ingreso.

| <b>Atributos</b>                                                                                                                   | <b>Métodos</b>                     |
|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| - idVisitante: int,<br>- nombres: String,<br>- apellidos: String,<br>- cedula_visit: String,<br>- antecedentesVerificados: boolean | + verificarAntecedentes(): boolean |

- ✓ **CartillaVisita:** Clase asociativa/entidad intermedia que formaliza la relación autorizada entre un Visitante y un Recluso.

| <b>Atributos</b>                                                                | <b>Métodos</b>                |
|---------------------------------------------------------------------------------|-------------------------------|
| - idCartilla: int,<br>- parentescoRol: String,<br>- estadoAutorizacion: boolean | + revocarAutorizacion(): void |

- ✓ **RegistroVisita:** Control de bitácora física de acceso.

| <b>Atributos</b>                                                                                            | <b>Métodos</b>                                           |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| - idVisita: int,<br>- fechaHoraEntrada: DateTime,<br>- fechaHoraSalida: DateTime,<br>- pertenencias: String | + registrarEntrada(): void,<br>+ registrarSalida(): void |

### Módulo 4: Gestión de Personal

- ✓ **Personal:** Empleados del penal.

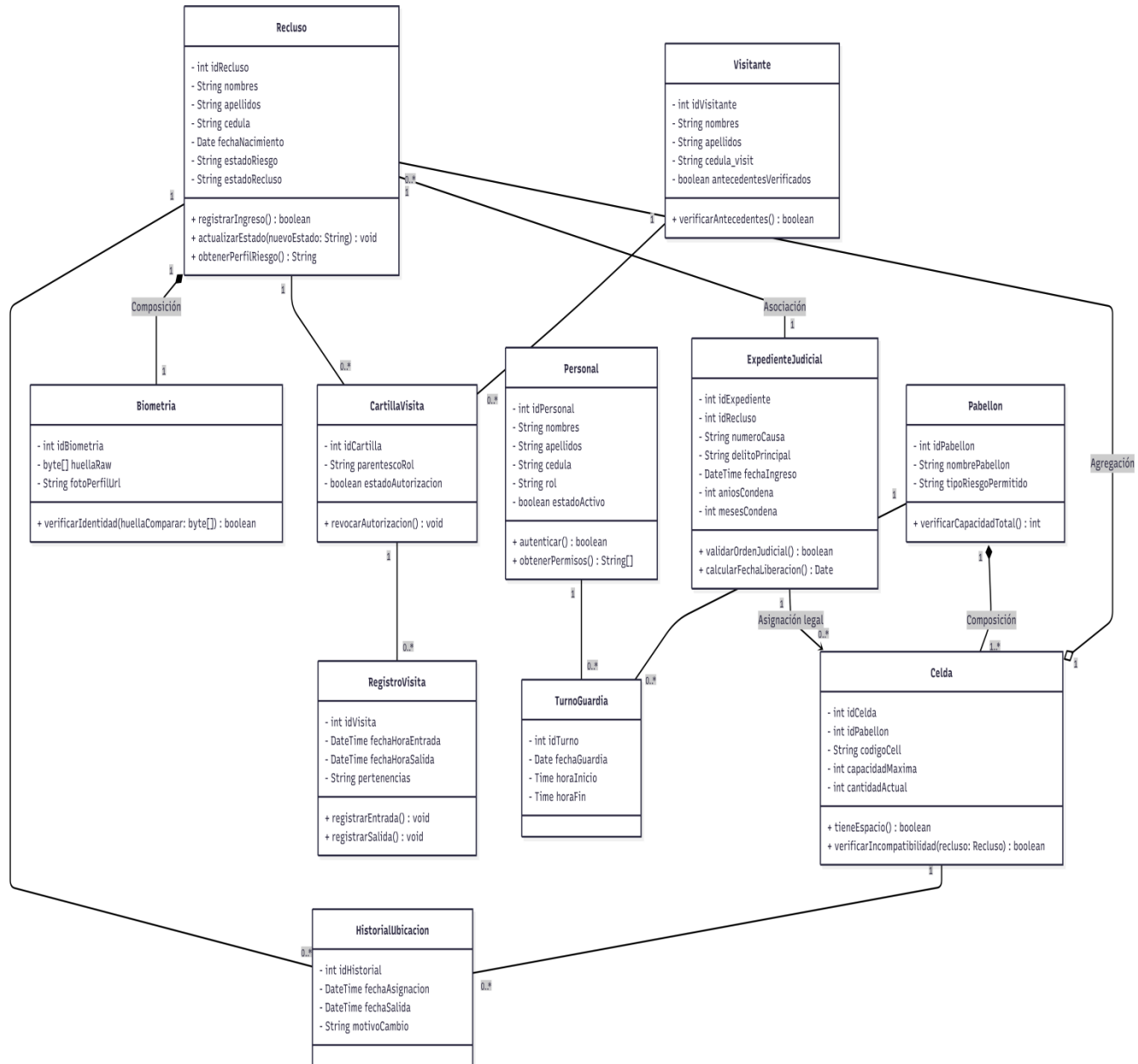
| <b>Atributos</b>                         | <b>Métodos</b>           |
|------------------------------------------|--------------------------|
| - idPersonal: int,<br>- nombres: String, | + autenticar(): boolean, |

|                                                                                                                                                            |                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- apellidos: String,</li> <li>- cedula: String,</li> <li>- rol: String,</li> <li>- estadoActivo: boolean</li> </ul> | <ul style="list-style-type: none"> <li>+ obtenerPermisos(): String[]</li> </ul> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|

- ✓ **TurnoGuardia:** Planificación de seguridad que asocia al Personal con un Pabellon específico.

| <b><i>Atributos</i></b>                                                                                                                                | <b><i>Métodos</i></b> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <ul style="list-style-type: none"> <li>- idTurno: int,</li> <li>- fechaGuardia: Date,</li> <li>- horaInicio: Time,</li> <li>- horaFin: Time</li> </ul> |                       |

# DIAGRAMA DE CLASES DEL SISTEMA DE GESTION PENITENCIARIO



## MODELO CONCEPTUAL (Entidades y Relaciones)

**Pabellón - Celda:** Un Pabellón se compone de una o muchas Celdas (**Composición 1 -> \***). Si desaparece el pabellón, desaparecen sus celdas.

**Celda - Recluso:** Una Celda puede albergar de cero a muchos Reclusos (**Agregación 1-> \***). El recluso existe independientemente de la celda.

**Recluso - Expediente:** Un Recluso tiene un único Expediente Judicial (**Asociación 1 -> 1**).

**Recluso - Visita:** Un Recluso puede recibir muchas Visitas, y un Visitante puede visitar a muchos Reclusos (Relación \* <-> \*)

**Personal - Turno:** El Personal Penal cumple Turnos en Áreas específicas del penal.

## 2. MODELO LÓGICO DE LA BASE DE DATOS

A continuación se presenta el diseño lógico del sistema de GP.

### Módulo 1: Registro e Ingreso (Admisión)

#### *Tabla: recluso*

id\_recluso (INT, PK, SERIAL)  
nombres (VARCHAR(100))  
apellidos (VARCHAR(100))  
cedula (VARCHAR(20), UNIQUE)  
fecha\_nacimiento (DATE)  
sexo (CHAR(1))  
estado\_riesgo (VARCHAR(15)) -- 'Bajo', 'Medio', 'Alto'  
estado\_recluso (VARCHAR(20)) -- 'En Ingreso', 'Activo',  
'Trasladado', 'Libertad', fecha\_ingreso (DATE)  
fecha\_salida (DATE)  
observaciones (TEXT)

#### *Tabla: expediente\_judicial*

- id\_expediente (INT, PK, SERIAL)
- id\_recluso (INT, FK -> recluso.id\_recluso, UNIQUE) -- Relación 1:1
- numero\_causa (VARCHAR(50), UNIQUE) -- Código del tribunal
- delito\_principal (VARCHAR(255))
- fecha\_ingreso (TIMESTAMP)
- anios\_condena (INT)
- meses\_condena (INT)
- orden\_judicial\_url (VARCHAR(255)) -- sera un enlace al documento legal escaneado.

***Tabla: biometria (si aplica, no es seguro)***

- id\_biometria (INT, PK, SERIAL)
- id\_recluso (INT, FK -> recluso.id\_recluso, UNIQUE) -- Relación 1:1
- huella\_dactilar\_raw (BYTEA o TEXT) -- Hash o vector de la huella
- foto\_perfil\_url (VARCHAR(255))
- fecha\_captura (TIMESTAMP) /de la biometria no del preso

**Módulo 2: Control de Ubicación en celda**

***Tabla: pabellon***

- id\_pabellon (INT, PK, SERIAL)
- nombre\_pabellon (VARCHAR(50), UNIQUE) -- Ej. 'Pabellón A - Máxima Seguridad'
- tipo\_riesgo\_permitido (VARCHAR(15)) -- Para validar con el estado del recluso

***Tabla: celda***

- id\_celda (INT, PK, SERIAL)
- id\_pabellon (INT, FK -> pabellon.id\_pabellon)
- codigo\_celda (VARCHAR(10)) -- Ej. 'A-01'
- capacidad\_maxima (INT)
- cantidad\_actual (INT, DEFAULT 0)

***Tabla: historial\_ubicacion***

- id\_historial (INT, PK, SERIAL)
- id\_recluso (INT, FK -> recluso.id\_recluso)
- id\_celda (INT, FK -> celda.id\_celda)
- fecha\_asignacion (TIMESTAMP)
- fecha\_salida (TIMESTAMP, NULLABLE)
- motivo\_cambio (TEXT)

**Módulo 3: Seguridad y Visitas**

***Tabla: visitante***

- id\_visitante (INT, PK, SERIAL)
- nombres (VARCHAR(100))
- apellidos (VARCHAR(100))
- Cedula\_visitante (VARCHAR(20), UNIQUE)
- antecedentes\_verificados (BOOLEAN, DEFAULT FALSE)

***Tabla: cartilla\_visita (Relación formal de parentesco con el preso)***

- id\_cartilla (INT, PK, SERIAL)
- id\_recluso (INT, FK -> recluso.id\_recluso)
- id\_visitante (INT, FK -> visitante.id\_visitante)
- parentesco\_rol (VARCHAR(30)) -- 'Madre', 'Hijo', 'Abogado', etc.
- estado\_autorizacion (BOOLEAN, DEFAULT TRUE)

***Tabla: registro\_visita (Control de Acceso)***

- id\_visita (INT, PK, SERIAL)
- id\_cartilla (INT, FK -> cartilla\_visita.id\_cartilla)
- fecha\_hora\_entrada (TIMESTAMP)
- fecha\_hora\_salida (TIMESTAMP, NULLABLE)
- pertenencias\_registradas (TEXT)

## **Módulo 4: Gestión de Personal y Seguridad.**

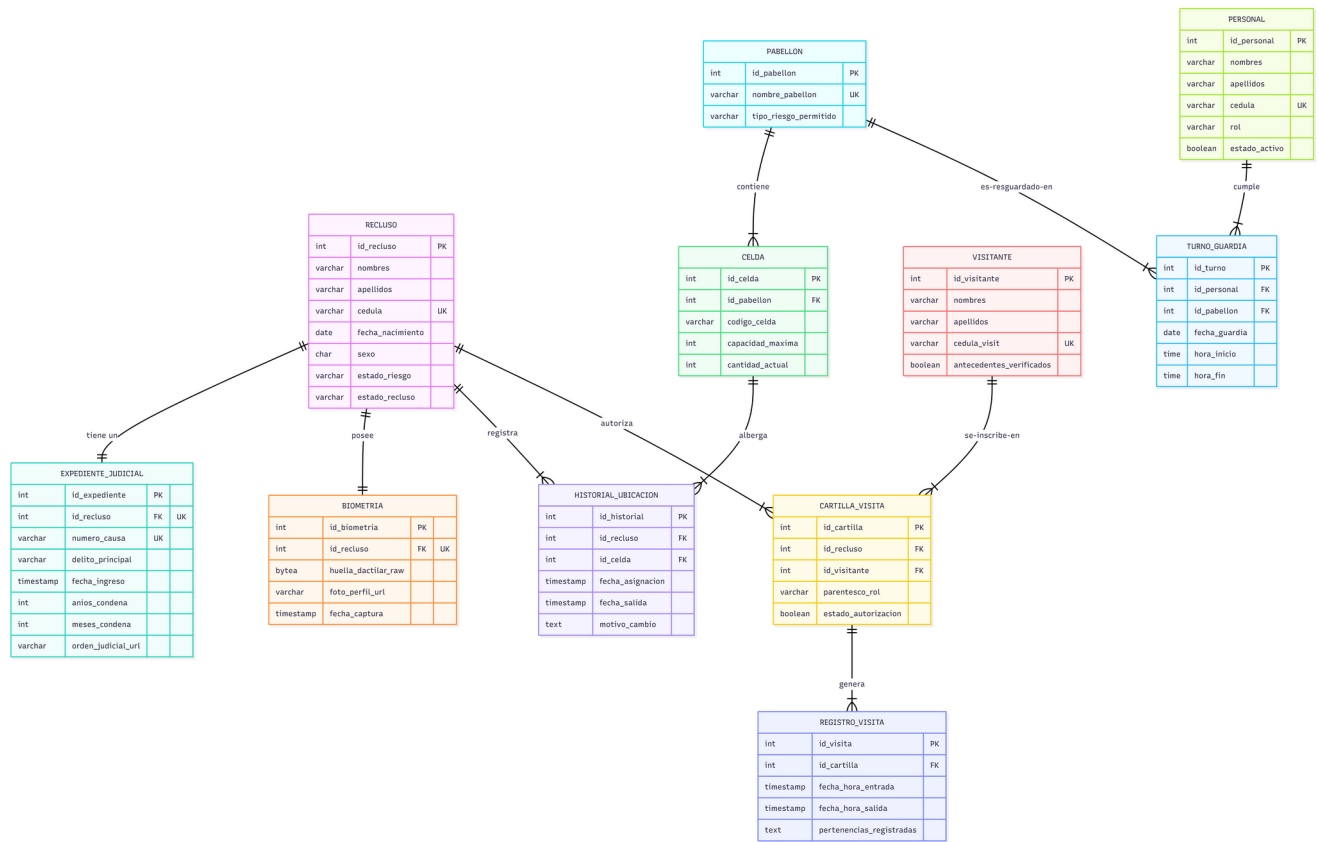
### ***Tabla: personal***

- id\_personal (INT, PK, SERIAL)
- nombres (VARCHAR(100))
- apellidos (VARCHAR(100))
- cedula (VARCHAR(20), UNIQUE)
- rol (VARCHAR(20)) -- 'Administrador', 'Custodio', 'Medico')
- estado\_activo (BOOLEAN, DEFAULT TRUE)

### ***Tabla: turno\_guardia***

- id\_turno (INT, PK, SERIAL)
- id\_personal (INT, FK -> personal.id\_personal)
- id\_pabellon (INT, FK -> pabellon.id\_pabellon) -- Área asignada
- fecha\_guardia (DATE)
- hora\_inicio (TIME)
- hora\_fin (TIME)

# Diagrama ER del Sistema de Gestión Penitenciaria



Modelo ER-. Fuente: Los Autores.



## SCRIPT DDL del Sistema de Gestion Penitenciaria.

```
--
=====
-- SISTEMA DE GESTIÓN PENITENCIARIA (SGP)
-- BASE DE DATOS OPTIMIZADA PARA POSTGRESQL
-- Yusvania Jimenez, Franklin Mundaray, Asdrúbal Hernández
=====

-- Eliminar tablas en orden inverso a las dependencias
DROP TABLE IF EXISTS turno_guardia CASCADE;
DROP TABLE IF EXISTS registro_visita CASCADE;
DROP TABLE IF EXISTS cartilla_visita CASCADE;
DROP TABLE IF EXISTS visitante CASCADE;
DROP TABLE IF EXISTS historial_ubicacion CASCADE;
DROP TABLE IF EXISTS celda CASCADE;
DROP TABLE IF EXISTS pabellon CASCADE;
DROP TABLE IF EXISTS biometria CASCADE;
DROP TABLE IF EXISTS expediente_judicial CASCADE;
DROP TABLE IF EXISTS recluso CASCADE;
DROP TABLE IF EXISTS personal CASCADE;

--
=====
-- MÓDULO 4: GESTIÓN DE PERSONAL (RBAC)
--
=====

CREATE TABLE personal (
    id_personal SERIAL PRIMARY KEY,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    cedula VARCHAR(20) NOT NULL UNIQUE,
    rol VARCHAR(20) NOT NULL,
    estado_activo BOOLEAN NOT NULL DEFAULT TRUE,
    fecha_creacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_personal_rol CHECK (rol IN ('Administrador', 'Custodio',
'Medico', 'JefeSeguridad', 'EncargadoAdmision')),
    CONSTRAINT chk_personal_cedula CHECK (cedula ~ '^[0-9]+$')
);

COMMENT ON TABLE personal IS 'Empleados del penal con control RBAC';
COMMENT ON COLUMN personal.rol IS 'Rol para control de acceso:
Administrador, Custodio, Medico, JefeSeguridad, EncargadoAdmision';

--
=====
-- MÓDULO 1: REGISTRO E INGRESO (ADMISIÓN)
--
=====
```

```

CREATE TABLE recluso (
  id_recluso SERIAL PRIMARY KEY,
  nombres VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  cedula VARCHAR(20) NOT NULL UNIQUE,
  fecha_nacimiento DATE NOT NULL,
  sexo CHAR(1) NOT NULL,
  estado_riesgo VARCHAR(15) NOT NULL,
  estado_recluso VARCHAR(20) NOT NULL DEFAULT 'En Ingreso',
  fecha_ingreso DATE NOT NULL DEFAULT CURRENT_DATE,
  fecha_salida DATE NULL,
  observaciones TEXT NULL,
  id_celda_actual INT NULL, -- FK a celda, se valida con trigger
  CONSTRAINT chk_recluso_sexo CHECK (sexo IN ('M', 'F', 'X')),
  CONSTRAINT chk_recluso_riesgo CHECK (estado_riesgo IN ('Bajo', 'Medio',
'Alto')),
  CONSTRAINT chk_recluso_estado CHECK (estado_recluso IN ('En Ingreso',
'Activo', 'Trasladado', 'Libertad', 'Fallecido')),
  CONSTRAINT chk_fechas CHECK (fecha_salida IS NULL OR fecha_salida >=
fecha_ingreso)
);

```

```

COMMENT ON TABLE recluso IS 'Información central del interno';
COMMENT ON COLUMN recluso.estado_riesgo IS 'Perfil de peligrosidad: Bajo,
Medio, Alto';
COMMENT ON COLUMN recluso.estado_recluso IS 'Estado actual en el sistema
penitenciario';

```

```

CREATE TABLE expediente_judicial (
  id_expediente SERIAL PRIMARY KEY,
  id_recluso INT NOT NULL UNIQUE,
  numero_causa VARCHAR(50) NOT NULL UNIQUE,
  delito_principal VARCHAR(255) NOT NULL,
  delitos_secundarios TEXT NULL,
  fecha_ingreso TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  anios_condena INT NOT NULL DEFAULT 0,
  meses_condena INT NOT NULL DEFAULT 0,
  orden_judicial_url VARCHAR(255) NOT NULL,
  juez VARCHAR(100) NULL,
  tribunal VARCHAR(100) NULL,
  CONSTRAINT fk_expediente_recluso FOREIGN KEY (id_recluso)
REFERENCES recluso(id_recluso) ON DELETE CASCADE,
  CONSTRAINT chk_expediente_tiempo CHECK (anios_condena >= 0 AND
meses_condena BETWEEN 0 AND 11),
  CONSTRAINT chk_expediente_orden CHECK (orden_judicial_url ~ '^https?:/'
OR orden_judicial_url ~ '^/')
);

```

```

COMMENT ON TABLE expediente_judicial IS 'Expediente legal del recluso -
relación 1:1 con recluso';

```

COMMENT ON COLUMN expediente\_judicial.orden\_judicial\_url IS 'URL o ruta del documento legal escaneado';

```
CREATE TABLE biometria (  
    id_biometria SERIAL PRIMARY KEY,  
    id_recluso INT NOT NULL UNIQUE,  
    huella_dactilar_raw BYTEA NOT NULL,  
    foto_perfil_url VARCHAR(255) NOT NULL,  
    fecha_captura TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    capturado_por INT NOT NULL,  
    CONSTRAINT fk_biometria_recluso FOREIGN KEY (id_recluso)  
REFERENCES recluso(id_recluso) ON DELETE CASCADE,  
    CONSTRAINT fk_biometria_capturador FOREIGN KEY (capturado_por)  
REFERENCES personal(id_personal)  
);
```

COMMENT ON TABLE biometria IS 'Datos biométricos para evitar suplantación de identidad - relación 1:1';

```
--  
=====
```

**-- MÓDULO 2: CONTROL DE UBICACIÓN EN CELDAS**

```
--  
=====
```

```
CREATE TABLE pabellon (  
    id_pabellon SERIAL PRIMARY KEY,  
    nombre_pabellon VARCHAR(50) NOT NULL UNIQUE,  
    tipo_riesgo_permitido VARCHAR(15) NOT NULL,  
    capacidad_total INT NOT NULL,  
    CONSTRAINT chk_pabellon_riesgo CHECK (tipo_riesgo_permitido IN ('Bajo',  
'Medio', 'Alto')),  
    CONSTRAINT chk_pabellon_capacidad CHECK (capacidad_total > 0)  
);
```

COMMENT ON TABLE pabellon IS 'Estructura física mayor que contiene celdas';  
COMMENT ON COLUMN pabellon.tipo\_riesgo\_permitido IS 'Solo permite reclusos con este nivel de peligrosidad';

```
CREATE TABLE celda (  
    id_celda SERIAL PRIMARY KEY,  
    id_pabellon INT NOT NULL,  
    codigo_celda VARCHAR(10) NOT NULL UNIQUE,  
    capacidad_maxima INT NOT NULL,  
    cantidad_actual INT NOT NULL DEFAULT 0,  
    activa BOOLEAN NOT NULL DEFAULT TRUE,  
    CONSTRAINT fk_celda_pabellon FOREIGN KEY (id_pabellon) REFERENCES  
pabellon(id_pabellon) ON DELETE RESTRICT,  
    CONSTRAINT chk_celda_capacidad CHECK (cantidad_actual <=  
capacidad_maxima),
```

```
    CONSTRAINT chk_celda_valores_positivos CHECK (capacidad_maxima > 0
AND cantidad_actual >= 0)
);
```

```
COMMENT ON TABLE celda IS 'Espacio de confinamiento individual o
compartido';
```

```
CREATE TABLE historial_ubicacion (
    id_historial SERIAL PRIMARY KEY,
    id_recluso INT NOT NULL,
    id_celda INT NOT NULL,
    fecha_asignacion TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    fecha_salida TIMESTAMP NULL,
    motivo_cambio TEXT NOT NULL,
    registrado_por INT NOT NULL,
    CONSTRAINT fk_historial_recluso FOREIGN KEY (id_recluso) REFERENCES
recluso(id_recluso) ON DELETE CASCADE,
    CONSTRAINT fk_historial_celda FOREIGN KEY (id_celda) REFERENCES
celda(id_celda) ON DELETE RESTRICT,
    CONSTRAINT fk_historial_registrador FOREIGN KEY (registrado_por)
REFERENCES personal(id_personal),
    CONSTRAINT chk_historial_fechas CHECK (fecha_salida IS NULL OR
fecha_salida >= fecha_asignacion)
);
```

```
COMMENT ON TABLE historial_ubicacion IS 'Auditoría de todos los movimientos
del recluso';
```

```
--
```

```
=====
```

```
-- MÓDULO 3: SEGURIDAD Y VISITAS
```

```
--
```

```
=====
```

```
CREATE TABLE visitante (
    id_visitante SERIAL PRIMARY KEY,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    cedula_visitante VARCHAR(20) NOT NULL UNIQUE,
    telefono VARCHAR(20) NULL,
    direccion TEXT NULL,
    antecedentes_verificados BOOLEAN NOT NULL DEFAULT FALSE,
    fecha_verificacion TIMESTAMP NULL,
    CONSTRAINT chk_visitante_cedula CHECK (cedula_visitante ~ '^[0-9]+$')
);
```

```
COMMENT ON TABLE visitante IS 'Persona externa autorizada a visitar reclusos';
```

```
CREATE TABLE cartilla_visita (
    id_cartilla SERIAL PRIMARY KEY,
```

```

    id_recluso INT NOT NULL,
    id_visitante INT NOT NULL,
    parentesco_rol VARCHAR(30) NOT NULL,
    estado_autorizacion BOOLEAN NOT NULL DEFAULT TRUE,
    fecha_autorizacion TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    fecha_revocacion TIMESTAMP NULL,
    registrado_por INT NOT NULL,
    CONSTRAINT fk_cartilla_recluso FOREIGN KEY (id_recluso) REFERENCES
recluso(id_recluso) ON DELETE CASCADE,
    CONSTRAINT fk_cartilla_visitante FOREIGN KEY (id_visitante) REFERENCES
visitante(id_visitante) ON DELETE CASCADE,
    CONSTRAINT fk_cartilla_registrador FOREIGN KEY (registrado_por)
REFERENCES personal(id_personal),
    CONSTRAINT uk_cartilla_recluso_visitante UNIQUE (id_recluso, id_visitante)
);

```

```

COMMENT ON TABLE cartilla_visita IS 'Relación formal entre visitante y recluso
(vínculo autorizado)';

```

```

CREATE TABLE registro_visita (
    id_visita SERIAL PRIMARY KEY,
    id_cartilla INT NOT NULL,
    fecha_hora_entrada TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    fecha_hora_salida TIMESTAMP NULL,
    pertenencias_registradas TEXT NULL,
    autorizado_por INT NOT NULL,
    observaciones_seguridad TEXT NULL,
    CONSTRAINT fk_registro_cartilla FOREIGN KEY (id_cartilla) REFERENCES
cartilla_visita(id_cartilla) ON DELETE RESTRICT,
    CONSTRAINT fk_registro_autorizador FOREIGN KEY (autorizado_por)
REFERENCES personal(id_personal),
    CONSTRAINT chk_registro_salida CHECK (fecha_hora_salida IS NULL OR
fecha_hora_salida >= fecha_hora_entrada)
);

```

```

COMMENT ON TABLE registro_visita IS 'Control de acceso físico de visitantes
(entrada/salida)';

```

```

--
=====
-- MÓDULO 4 (CONTINUACIÓN): TURNOS Y ASIGNACIÓN DE GUARDIA
--
=====

```

```

CREATE TABLE turno_guardia (
    id_turno SERIAL PRIMARY KEY,
    id_personal INT NOT NULL,
    id_pabellon INT NOT NULL,
    fecha_guardia DATE NOT NULL,

```

```

    hora_inicio TIME NOT NULL,
    hora_fin TIME NOT NULL,
    asignado_por INT NOT NULL,
    CONSTRAINT fk_turno_personal FOREIGN KEY (id_personal) REFERENCES
personal(id_personal) ON DELETE RESTRICT,
    CONSTRAINT fk_turno_pabellon FOREIGN KEY (id_pabellon) REFERENCES
pabellon(id_pabellon) ON DELETE RESTRICT,
    CONSTRAINT fk_turno_asignador FOREIGN KEY (asignado_por)
REFERENCES personal(id_personal),
    CONSTRAINT chk_turno_horario CHECK (hora_fin > hora_inicio)
);

```

```

COMMENT ON TABLE turno_guardia IS 'Planificación de horarios de custodios por
área';

```

```

--
=====
-- CREACIÓN DE ÍNDICES OPTIMIZADOS
--
=====

```

```

-- Índices para búsquedas frecuentes
CREATE INDEX idx_recluso_buscar ON recluso(apellidos, nombres);
CREATE INDEX idx_recluso_cedula ON recluso(cedula);
CREATE INDEX idx_recluso_estado ON recluso(estado_recluso);
CREATE INDEX idx_recluso_riesgo ON recluso(estado_riesgo);
CREATE INDEX idx_recluso_celda_actual ON recluso(id_celda_actual);

CREATE INDEX idx_expediente_causa ON expediente_judicial(numero_causa);
CREATE INDEX idx_expediente_recluso ON expediente_judicial(id_recluso);

CREATE INDEX idx_historial_activo ON historial_ubicacion(id_recluso) WHERE
fecha_salida IS NULL;
CREATE INDEX idx_historial_recluso_fecha ON historial_ubicacion(id_recluso,
fecha_asignacion);

CREATE INDEX idx_visita_fecha ON registro_visita(fecha_hora_entrada);
CREATE INDEX idx_visita_cartilla ON registro_visita(id_cartilla);

CREATE INDEX idx_cartilla_recluso ON cartilla_visita(id_recluso);
CREATE INDEX idx_cartilla_visitante ON cartilla_visita(id_visitante);

CREATE INDEX idx_turno_fecha ON turno_guardia(fecha_guardia);
CREATE INDEX idx_turno_personal ON turno_guardia(id_personal);

-- Índice para validar capacidad rápida de celdas
CREATE INDEX idx_celda_ocupacion ON celda(id_pabellon, cantidad_actual);

--
=====
-- FUNCIONES Y TRIGGERS PARA REGLAS DE NEGOCIO

```

```

--
=====

-- 1. Trigger para evitar mezcla de reclusos con perfiles incompatibles
CREATE OR REPLACE FUNCTION validar_compatibilidad_recluso_celda()
RETURNS TRIGGER AS $$
DECLARE
    v_tipo_riesgo_celda VARCHAR(15);
    v_banda_recluso_nueva VARCHAR(50);
    v_banda_recluso_existente VARCHAR(50);
BEGIN
    -- Obtener el tipo de riesgo permitido del pabellón de la celda
    SELECT p.tipo_riesgo_permitido INTO v_tipo_riesgo_celda
    FROM celda c
    JOIN pabellon p ON c.id_pabellon = p.id_pabellon
    WHERE c.id_celda = NEW.id_celda;

    -- Verificar que el recluso tenga el perfil de riesgo adecuado
    IF (SELECT estado_riesgo FROM recluso WHERE id_recluso = NEW.id_recluso)
    != v_tipo_riesgo_celda THEN
        RAISE EXCEPTION 'No se puede asignar: El recluso de riesgo % no puede ir a
        celda en pabellón de riesgo %',
        (SELECT estado_riesgo FROM recluso WHERE id_recluso =
        NEW.id_recluso), v_tipo_riesgo_celda;
    END IF;

    -- Verificar que no haya reclusos de bandas rivales en la misma celda
    -- Asumiendo que tenemos una tabla 'banda' relacionada con recluso
    -- Esta validación es extensible según necesidades específicas

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Nota: El trigger requiere que la tabla recluso tenga id_celda_actual
-- y se actualice consistentemente

-- 2. Función para actualizar cantidad_actual en celda
CREATE OR REPLACE FUNCTION actualizar_ocupacion_celda()
RETURNS TRIGGER AS $$
BEGIN
    -- Al asignar un recluso (nuevo registro activo en historial)
    IF TG_OP = 'INSERT' AND NEW.fecha_salida IS NULL THEN
        UPDATE celda
        SET cantidad_actual = cantidad_actual + 1
        WHERE id_celda = NEW.id_celda;

        -- Actualizar id_celda_actual en recluso
        UPDATE recluso
        SET id_celda_actual = NEW.id_celda
        WHERE id_recluso = NEW.id_recluso;
    
```

```

-- Al liberar un recluso (fecha_salida se actualiza de NULL a valor)
ELSIF TG_OP = 'UPDATE' AND OLD.fecha_salida IS NULL AND
NEW.fecha_salida IS NOT NULL THEN
    UPDATE celda
    SET cantidad_actual = cantidad_actual - 1
    WHERE id_celda = OLD.id_celda;

    -- Limpiar id_celda_actual en recluso
    UPDATE recluso
    SET id_celda_actual = NULL
    WHERE id_recluso = NEW.id_recluso;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Crear trigger en historial_ubicacion
DROP TRIGGER IF EXISTS trg_actualizar_ocupacion ON historial_ubicacion;
CREATE TRIGGER trg_actualizar_ocupacion
AFTER INSERT OR UPDATE ON historial_ubicacion
FOR EACH ROW
EXECUTE FUNCTION actualizar_ocupacion_celda();

-- 3. Trigger para controlar aforo de visitas (evitar hacinamiento)
CREATE OR REPLACE FUNCTION validar_aforo_visitas()
RETURNS TRIGGER AS $$
DECLARE
    v_conteo_activo INT;
    v_aforo_maximo INT := 50; -- Configurable según sala de visitas
BEGIN
    -- Contar visitas activas (sin salida registrada) en el mismo día
    SELECT COUNT(*) INTO v_conteo_activo
    FROM registro_visita rv
    JOIN cartilla_visita cv ON rv.id_cartilla = cv.id_cartilla
    WHERE DATE(rv.fecha_hora_entrada) = DATE(NEW.fecha_hora_entrada)
    AND rv.fecha_hora_salida IS NULL;

    IF v_conteo_activo >= v_aforo_maximo THEN
        RAISE EXCEPTION 'Aforo máximo de visitas alcanzado para hoy (%)',
v_aforo_maximo;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_validar_aforo ON registro_visita;
CREATE TRIGGER trg_validar_aforo
BEFORE INSERT ON registro_visita

```



```

FOR EACH ROW
EXECUTE FUNCTION validar_aforo_visitas();

-- 4. Función para registrar automáticamente el historial al cambiar celda
CREATE OR REPLACE FUNCTION registrar_cambio_celda()
RETURNS TRIGGER AS $$
BEGIN
    -- Si cambió la celda actual del recluso
    IF OLD.id_celda_actual IS DISTINCT FROM NEW.id_celda_actual THEN
        -- Cerrar historial activo anterior
        UPDATE historial_ubicacion
        SET fecha_salida = CURRENT_TIMESTAMP,
            motivo_cambio = 'Traslado manual por actualización'
        WHERE id_recluso = NEW.id_recluso
            AND fecha_salida IS NULL;

        -- Crear nuevo registro si la nueva celda no es NULL
        IF NEW.id_celda_actual IS NOT NULL THEN
            INSERT INTO historial_ubicacion (id_recluso, id_celda, motivo_cambio)
            VALUES (NEW.id_recluso, NEW.id_celda_actual, 'Asignación automática
por actualización de celda');
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_registrar_cambio_celda ON recluso;
CREATE TRIGGER trg_registrar_cambio_celda
AFTER UPDATE OF id_celda_actual ON recluso
FOR EACH ROW
WHEN (OLD.id_celda_actual IS DISTINCT FROM NEW.id_celda_actual)
EXECUTE FUNCTION registrar_cambio_celda();

--
=====
-- DATOS DE PRUEBA (OPCIONAL)
--
=====

-- Insertar roles de personal base
INSERT INTO personal (nombres, apellidos, cedula, rol) VALUES
('Admin', 'Sistema', '999999999', 'Administrador');

-- Insertar pabellones de ejemplo
INSERT INTO pabellon (nombre_pabellon, tipo_riesgo_permitido, capacidad_total)
VALUES
('Pabellón A - Máxima Seguridad', 'Alto', 100),
('Pabellón B - Seguridad Media', 'Medio', 150),
('Pabellón C - Baja Seguridad', 'Bajo', 200);

```

```
-- Insertar celdas de ejemplo
INSERT INTO celda (id_pabellon, codigo_celda, capacidad_maxima) VALUES
(1, 'A-01', 2),
(1, 'A-02', 2),
(2, 'B-01', 4),
(2, 'B-02', 4),
(3, 'C-01', 6),
(3, 'C-02', 6);

--
=====
-- FIN DEL SCRIPT
--
=====
```